第29课 递归思想（一）

一、课程信息

项目	内容
课次	第 29 课 / 共 40 课
主题	递归思想（一）——函数自己调用自己
前置知识	函数的定义、调用、参数与返回值（第13课）
后续衔接	第30课 递归思想（二）——汉诺塔、阶乘的更多变体

教学目标（可检验）

1. **能说出递归三要素**：做什么、递归调用（缩小问题）、终止条件，并在给定代码中标注出来。
2. **能手写两个基础递归函数**：倒计时（从 N 数到 1）和求 1+2+…+N。
3. **能画出简单递归的调用栈示意图**（3～5 层），并说明缺少终止条件会导致栈溢出。

二、教学重点与难点

重点

- 递归的概念：函数在自己的身体里调用自己。
- 递归三要素的识别与编写。
- 用调用栈图理解递归的"去"和"回"两个过程。

难点

- 理解"函数还没执行完就又调用了自己"——执行流程的跳转。
- 终止条件的设计：什么时候该停下来？
- 从"循环思维"转向"递归思维"：把大问题拆成"做一点 + 剩下的交给自己"。

三、课堂时间分配（共 100 分钟）

时间	环节	内容	形式
0′–5′ (5 min)	导入	俄罗斯套娃故事 + 照镜子实验	讲述+互动
5′–20′ (15 min)	讲解①	递归概念、三要素、倒计时例子	讲解+板书画图
20′–30′ (10 min)	练习①	学生动手写倒计时递归、画调用栈图	上机+纸笔
30′–35′ (5 min)	讨论	如果忘了终止条件会怎样？演示栈溢出	演示+讨论
35′–55′ (20 min)	讲解②	例题：求和 1+2+…+N、正序输出数字	讲解+画图
55′–75′ (20 min)	练习②	课堂练习题（2～3 题上机）	上机实操
75′–90′ (15 min)	讲解③	例题：递归判断回文串（拓展）	讲解+画图
90′–97′ (7 min)	总结	回顾三要素、常见错误、学生提问	互动
97′–100′ (3 min)	布置	课后作业 + 下节课预告	讲述

> **说明**：三段讲解分别为 15 min、20 min、15 min，均不超过 25 分钟，中间穿插动手练习。

四、讲稿提纲

4.1 导入——俄罗斯套娃的故事（5 min）

> 同学们，老师今天带来了一个"俄罗斯套娃"。你们玩过吗？
>
> 打开最外面的大娃娃——哇，里面还有一个小一点的娃娃！
> 再打开——里面还有一个更小的！
> 再打开——又有一个！
> 一直打开……直到最后那个**最小的实心娃娃**，打不开了，就停下来了。
>
> 这就是我们今天要学的新本领——**递归**。
>
> 递归就像套娃：**每一层做的事情都一样**（打开娃娃），但**娃娃越来越小**（问题越来越简单），直到碰到**最小的那个娃娃**（终止条件），就不用再打开了。
>
> 再想一个画面：你站在两面平行的镜子中间，看一面镜子——里面有你，你里面还有你，还有你……无穷无尽！如果没有"最小的娃娃"来喊停，递归也会像这面镜子一样停不下来，电脑就崩溃了。

4.2 递归的概念（5 min）

> 在编程里，**递归**就是——一个函数在自己身体里面**调用自己**。
>
> 听起来很奇怪对不对？函数怎么能调自己呢？我们来看一个最简单的例子。

```
void 打开套娃(int 娃娃编号) {
 if (娃娃编号 == 最小的) {
 cout << "到底了，不用再打开了！" << endl;
 return; // 停下来!
 }
 cout << "打开第" << 娃娃编号 << "个娃娃" << endl;
 打开套娃(娃娃编号 + 1); // 打开里面那个更小的
}
```

> 看！`打开套娃` 这个函数里面，又调用了 `打开套娃`，只不过娃娃编号变大了（娃娃变小了）。这就是递归！

4.3 递归三要素（10 min，重点板书）

> 写递归函数，要记住三个宝贝，一个都不能少：

编号	要素	套娃比喻	作用
①	**做什么**（当前这一层要干的事）	打开眼前这个娃娃	处理当前问题
②	**递归调用**（把问题缩小，交给自己）	去打开里面更小的娃娃	让问题越来越简单
③	**终止条件**（最小的娃娃，该停了）	碰到实心的小娃娃就停	防止无穷无尽

> **口诀**：做事、缩小、喊停——三步写递归！

4.4 倒计时例子——从 N 数到 1（板书+画图）

> 想象老师说："从5开始倒数！"
> 你说：5！然后让旁边的同学从4开始倒数。
> 旁边的同学说：4！又让他旁边的从3开始倒数。
> ……一直到有人数到1，说"1！结束了！"就停下来。

画调用栈图（板书核心，慢慢画，每一步停下来问学生）：

```
countdown(5) 输出5，调用 countdown(4)
 countdown(4) 输出4，调用 countdown(3)
 countdown(3) 输出3，调用 countdown(2)
 countdown(2) 输出2，调用 countdown(1)
 countdown(1) 输出1，n==1 到底了，return！
 countdown(2) 回来了，结束
 countdown(3) 回来了，结束
 countdown(4) 回来了，结束
countdown(5) 回来了，结束
```

> 看到了吗？像搭积木一样，一层一层叠上去，到底之后再一层一层撤回来。这个"叠积木"的过程，计算机里叫做**调用栈**。

4.5 没有终止条件会怎样？（演示+讨论）

> 如果最小的套娃也能打开，里面还有更小的，永远打不完——会怎样？
> 电脑的调用栈就像一个有限高度的积木塔。如果一直叠叠叠，叠到天花板——"轰"一声塌了！
> 这就叫**栈溢出**（Stack Overflow），电脑会报错崩溃。
> 所以，**终止条件**是递归的安全网，绝对不能忘！

（现场演示一个没有终止条件的递归程序，让学生看到报错信息。）

4.6 求和 1+2+…+N（讲解②核心）

> 老师问你：1+2+3+4+5 等于多少？
> 你可以这样想：
> - 1+2+3+4+5 = **5** + (1+2+3+4)
> - 1+2+3+4 = **4** + (1+2+3)
> - 1+2+3 = **3** + (1+2)
> - 1+2 = **2** + (1)
> - 1 就是 1，不用再拆了！（终止条件）
>
> 发现了吗？`sum(n) = n + sum(n-1)`，这就是递归公式！
> 最小的情况是`sum(1) = 1`，这就是终止条件！

七、常见错误与应对

错误 1：忘记写终止条件

```
void countdown(int n) {
 cout << n << endl;
 countdown(n - 1); // 没有 if 判断，永远不会停！
}
```

现象：程序疯狂输出，然后崩溃（Segmentation Fault 或 Stack Overflow）。

应对：告诉学生口诀——"写递归，第一件事就是写终止条件！先写 if，再写别的。"

错误 2：终止条件写错，差了一步

```
int sum(int n) {
 if (n == 0) return 0; // 应该是 n==1 return 1（如果题目从1开始）
 return n + sum(n - 1);
}
```

现象：如果 n==0 return 0 其实也能正确工作（`sum(1) = 1 + sum(0) = 1 + 0 = 1`），但如果输入了负数就会栈溢出。

应对：让学生养成习惯——写完后用小数据（n=1, n=2）手动模拟一遍，验证终止条件是否正确。

错误 3：递归调用时问题没有缩小

```
void test(int n) {
 if (n == 0) return;
 cout << n << endl;
 test(n); // 参数没变！永远是 n，永远不会到 0!
}
```

现象：同一个数字无限输出，然后崩溃。

应对：提醒学生检查——"递归调用的参数有没有变小（或变简单）？如果没变，套娃永远一样大，打不完！"

错误 4：递归方向搞反

```
void countdown(int n) {
 if (n == 0) return;
 countdown(n - 1); // 先递归
 cout << n << endl; // 后输出
}
// 本想输出 5 4 3 2 1，结果输出了 1 2 3 4 5
```

现象：输出顺序和预期相反。

应对：画调用栈图！让学生搞清楚"先递归再做事"和"先做事再递归"的输出顺序是反的。这恰好也是练习1的考点。

错误 5：返回值忘记接收

```
int sum(int n) {
 if (n == 1) return 1;
 sum(n - 1); // 调用了，但没有 return！
}
```

现象：函数返回了随机数（未定义行为）。

应对：有返回值的递归函数，递归调用前面一定要写 `return`。

九、下节课预告

> 同学们，今天我们认识了递归这个强大的新朋友，学会了用"套娃"的方式让函数自己调用自己。
>
> **下节课——第30课《递归思想（二）》**，我们要挑战两个经典问题：
>
> 1. **汉诺塔**：三根柱子、N 个盘子，怎么搬？这是递归最经典、最优美的例子！
> 2. **阶乘的更多玩法**：用递归解决更多有趣的数学问题。
>
> 回去好好消化今天的内容，把倒计时和求和的代码多写几遍，画画调用栈图，下节课我们就能更轻松地理解汉诺塔啦！

教案结束